

Cache-First Pattern

 systemdesignschool.io/fundamentals/cache-always



Database optimization improves query performance through indexing and sharding. But even optimized databases cannot handle 100 million daily active users reading directly. The cache-first pattern solves this by placing fast storage between users and databases and checking cache before querying the database.

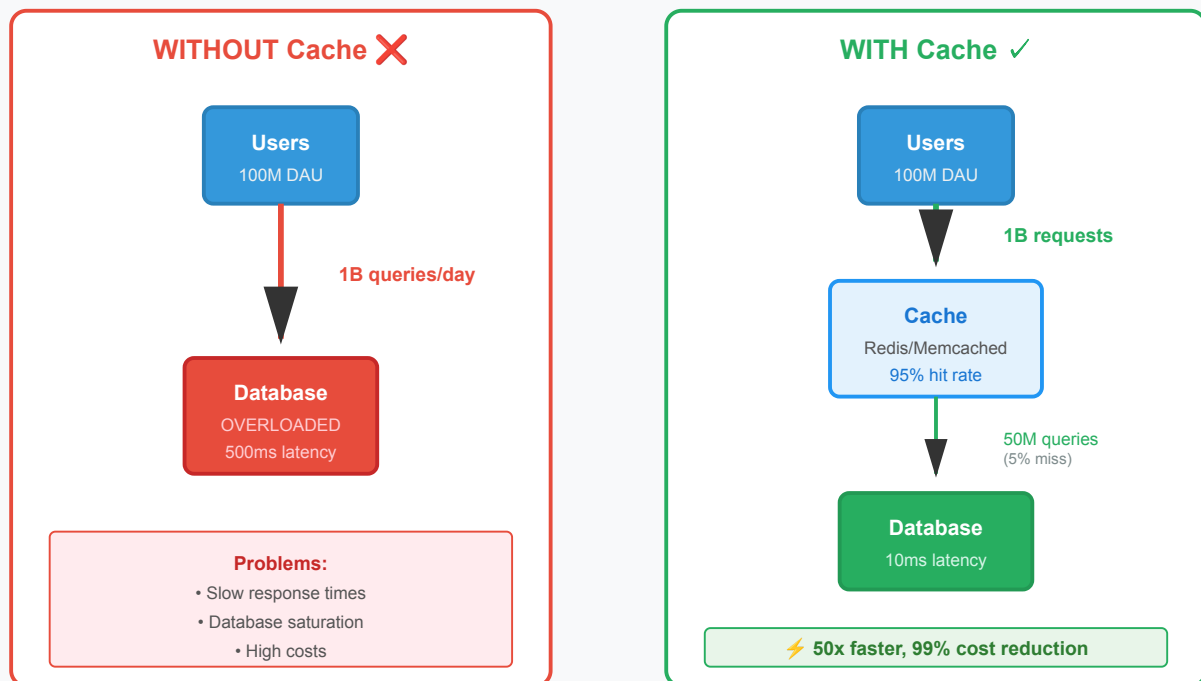
Reading from a database takes milliseconds. Reading from cache takes microseconds. At scale, this difference determines whether a system survives traffic spikes or collapses under load.

Why Cache Aggressively

In-memory caches like Redis or Memcached retrieve data hundreds of times faster than database queries. A product page that loads in roughly 50ms with cache might take 500ms querying the database directly. Users notice the delay. Conversion rates drop.

Caches also protect databases from overload. Consider a viral news article generating 100,000 requests per second. Without caching, those requests hit the database directly. The database saturates and query times spike by orders of magnitude. With caching, the first request fetches from the database and stores in cache. The next 99,999 requests hit cache. The database serves one query instead of 100,000.

Cache-Always Pattern



Cost optimization follows performance. Databases charge per read operation. A popular API endpoint serving 10 million requests per day costs real money at database pricing. With a 99% hit rate, the database handles only 100,000 cache misses instead of 10 million reads—a proportional drop in per-read costs.

Scale Requirements

At 100 million daily active users, direct database reads become impossible. Consider a social media feed. Each user loads their feed 10 times per day. That's 1 billion feed requests. Each feed aggregates posts from 500 friends. Without caching, the database executes 1 billion complex aggregation queries. No database survives this.

With caching, the system pre-computes feeds or caches query results. Most requests hit cache. Database load drops to manageable levels. The cache absorbs traffic spikes during peak hours. When a celebrity posts and their 10 million followers refresh feeds, cache serves the content without touching the database.

The economics make caching mandatory. Database clusters are priced per compute and I/O—scaling to high QPS is expensive. In-memory caches serve orders of magnitude more operations per dollar because reads from RAM avoid disk and query-planning overhead. The cache hit rate directly determines total infrastructure cost.

What to Cache

API responses cache well when they change infrequently. A product catalog updates hourly but gets queried thousands of times per second. Cache the catalog with a 1-hour TTL. One database query per hour replaces thousands.

Session data belongs in cache. User login state, shopping cart contents, and temporary preferences need fast access. Storing sessions in cache eliminates database queries on every page load. A user browsing 20 pages generates 20 cache lookups instead of 20 database queries.

Database query results cache effectively for repeated queries. A popular blog post generates identical queries as different users view it. Cache the rendered post for 5 minutes. Thousands of readers share one database query.

Computed values benefit from caching. Recommendation algorithms, trending topics, and aggregation results take seconds to compute but stay valid for minutes. Computing once and caching saves CPU and reduces latency.

Cache Strategies

Time-to-live expiration sets automatic invalidation. A news article caches for 5 minutes. After expiration, the next request fetches fresh data. This balances freshness and cache hit rate. Shorter TTLs keep data current. Longer TTLs reduce database load.

Write-through caching updates cache and database simultaneously. When a user updates their profile, write to both. This keeps cache consistent but adds write latency. Use write-through for critical data that must stay synchronized.

Write-behind caching updates cache immediately and database asynchronously. User profile changes appear instantly in cache. A background process persists to database within seconds. This improves write latency but risks data loss if the cache crashes before persistence.

Cache-aside lets the application manage cache explicitly. On read, check cache first. If missing, query database and populate cache. On write, update database and invalidate cache. This provides maximum control but requires careful implementation.

When Not to Cache

Real-time sensor data streaming at high frequency should not cache. Industrial automation systems reading sensors 1,000 times per second need current values. Cache adds latency without benefit. Push-based systems work better.

Financial market data updating multiple times per second rarely benefits from caching. Traders need the latest price. Caching for even 100ms shows stale data. Direct database reads or streaming updates suit this use case better.

Robotics and industrial control systems require immediate response. Caching introduces delays that affect safety and precision. These specialized domains operate outside typical web service patterns.

For web services, APIs, and user-facing applications serving millions of users, caching is mandatory. The performance gain and cost reduction justify the complexity.

Interview Application

When designing a system for 100 million users, suggest caching immediately. Explain that direct database reads cannot scale to this level. Mention specific cache placement: cache feeds, cache API responses, cache session data.

Discuss cache invalidation strategy. For a social network, mention that posts cache with short TTL because they change as users add likes and comments. Profile data caches longer because it changes less frequently. Show awareness that different data has different caching needs.

Mention cache-aside pattern by default. Application checks cache, falls back to database on miss, and populates cache. This gives control over what to cache and when to invalidate. Note that other patterns exist but cache-aside fits most scenarios.

Discuss cache hit rate. A 95% hit rate means only 5% of requests hit the database. With 10 million requests per day, that's 500,000 database queries instead of 10 million. Quantifying the improvement demonstrates understanding of the pattern's value.